Programación por bloques (Scratch) frente a programación textual en el aprendizaje de estructuras secuenciales

Yordy Pineda Jimenez
Loja - Ecuador
Computacion Primer Ciclo
Universidad Nacional de Loja
ORCID: https://orcid.org/0009-0008-4372-606X
Correo electrónico: yorpin206@gmail.com

RESUMEN

La programación basada en bloques se ha convertido en una herramienta importante para la enseñanza de la informática, especialmente en las primeras etapas del aprendizaje, debido a que facilita la comprensión de la lógica algorítmica mediante elementos visuales e interactivos. A diferencia de la programación textual, permite desarrollar algoritmos sin tener que dominar inicialmente reglas estrictas de sintaxis, reduciendo las dificultades de los principiantes. Herramientas como Scratch, Blockly y Snap! han contribuido a que los estudiantes se familiaricen con los conceptos fundamentales de programación.

Aunque se evidencia que el aprendizaje mediante bloques favorece la comprensión de estructuras de control y facilita la transición hacia lenguajes textuales, esta transición presenta dificultades relacionadas con la sintaxis, la escritura del código y la percepción de que los entornos visuales son menos profesionales. Ante estas limitaciones, surgen alternativas como la edición basada en marcos, que combina elementos visuales con la escritura de código textual. Ambas modalidades cumplen funciones importantes en la formación informática. Por ello, resulta conveniente implementar estrategias educativas progresivas y adaptativas que integren ambos enfoques.

***Index Terms*—Programación por bloques, programación textual, Scratch, informática educativa, educación secundaria.**

I. INTRODUCCIÓN

La programación es un campo que ha crecido enormemente en los últimos años pero enfrenta desafíos de aprendizaje complejos, desde entornos estrictamente textuales y sintácticos hasta entornos virtuales. La lógica que se emplea para poder diseñar un algoritmo genera retos en la enseñanza tanto para las personas que buscan aprender como para las que enseñan. Por ende, en respuesta a esta necesidad, ha crecido un área clave: "la programación basada en bloques es cada vez más la forma en que los estudiantes se introducen a la práctica de la programación y al campo de la informática" [1].

¿Cuál es la diferencia entre la programación de bloques y la programación textual?

La principal diferencia de la programación de bloques respecto a la textual reside en la ayuda visual que proporciona al estudiante, ya que permite aprender de manera más interactiva, desarrollando más fácilmente habilidades de comprensión de la lógica que existe detrás de un programa. La textual es más directa y menos intuitiva, haciendo que el aprendizaje dependa más de la forma en que se enseñe y del nivel de comprensión del estudiante. Aunque la programación textual busca cumplir el mismo propósito de comprender y entender cómo programar, presenta más complicaciones por su alto contenido de información en pantalla, la cual muchas veces satura al estudiante en lugar de guiarlo.

Aunque la programación por bloques no es una innovación reciente, en los últimos años ha tomado fuerza con la aparición de una nueva generación de herramientas encabezada por Scratch, Snap! y Blockly. Estas herramientas forman parte de un grupo amplio de editores llamados editores estructurados [2].

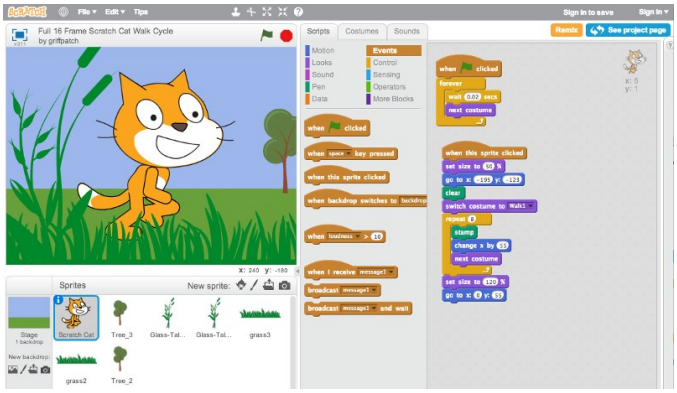


Figura 1. Interfaz del entorno de programación Scratch.

Scratch es uno de los lenguajes de programación visual más populares. Utiliza una interfaz basada en bloques que se ensamblan entre sí como un rompecabezas, sin necesidad de escribir código tradicional basado en largas líneas de texto. "Desde su lanzamiento, más de 35 millones de usuarios han creado cuentas en el sitio web de Scratch y se han compartido

casi 40 millones de proyectos; la mayoría de los usuarios tiene menos de 14 años" [1].

II. Resultados

Este ensayo se realiza a partir de la investigación de 5 artículos científicos. El trabajo de David Weintrop [1] da una perspectiva para examinar el papel de los entornos de programación basados en bloques (como Scratch y las iniciativas de Code.org) en la educación informática. El autor analiza cómo estas herramientas benefician a los principiantes, evalúa sus limitaciones y plantea el impacto que tendrán con el tiempo y el avance de la era digital. De manera complementaria, Weintrop y Wilensky [2], mediante un estudio práctico cuasiexperimental de 5 semanas, comparan de forma directa el impacto de utilizar una interfaz basada en bloques frente a una basada en texto en una clase introductoria de informática en secundaria.

Por otra parte, Fahid et al. [3] presentan un nuevo marco para modelar el aprendizaje y la resolución de problemas de 149 estudiantes universitarios en un entorno de programación basada en bloques. Utilizan la distancia de edición de árboles de sintaxis abstracta (AST) para medir la similitud incremental entre el código del estudiante y las soluciones de expertos. Asimismo, Gomez, Moresi y Benotti [4] evalúan el impacto de la experiencia previa en programación por bloques (Scratch) sobre la capacidad de niños de primaria (10-11 años) para aprender un lenguaje imperativo basado en texto, comparando una escuela con 3 semestres de experiencia en Scratch frente a otra sin experiencia previa.

Finalmente, Kölling, Brown y Altadmri [5] identifican la abrupta brecha técnica y cognitiva que existe entre la programación basada en bloques y la basada en texto, y proponen un nuevo paradigma de interacción llamado *Edición basada en marcos* (Frame-based editing), implementado a través del lenguaje Stride dentro del entorno educativo Greenfoot.

III. Discusión de Hallazgos

Según lo evidenciado por los textos, existe una clara convergencia respecto a los beneficios de la programación en bloques en las primeras etapas del aprendizaje. Como sostienen Weintrop [1] y Weintrop y Wilensky [2], la eliminación de la sobrecarga sintáctica permite que los estudiantes se concentren en la lógica algorítmica. Esto concuerda con las observaciones de Gomez et al. [4] en educación primaria, donde la experiencia previa en entornos visuales redujo los errores semánticos al empezar a escribir código textual.

Sin embargo, podemos apreciar un problema relacionado con el sentido de autenticidad percibida y el paso a lenguajes más profesionales. Aunque la modalidad de bloques es más rápida en términos de aprendizaje inmediato, Weintrop [1] señala que, en periodos de seguimiento (por ejemplo en Java), los beneficios de aprendizaje tienden a igualarse con los de aquellos que iniciaron directamente en texto. Esto se explica en parte por la resistencia de los estudiantes, quienes suelen percibir los bloques como una herramienta "no real" o infantil [1], [2].

Para resolver esta brecha cognitiva y técnica, propuestas como la edición basada en marcos descrita por Kölling et al. [5] ofrecen una alternativa intermedia viable. Al estructurar el código mediante recuadros gráficos pero manteniendo la escritura textual de expresiones, se facilita una transición escalonada que minimiza problemas clásicos de sintaxis y mecanografía sin abandonar la estructura rigurosa del lenguaje objetivo [5].

Finalmente, la efectividad de cualquiera de estas modalidades depende en gran medida del perfil y la trayectoria del estudiante. El modelado predictivo desarrollado por Fahid et al. [3] demuestra que la persistencia y la velocidad de avance varían drásticamente entre alumnos, permitiendo predecir patrones de abandono o éxito temprano. En consecuencia, la discusión actual no debe centrarse en sustituir el texto por bloques, sino en diseñar entornos híbridos adaptativos que acompañen al estudiante según su ritmo y nivel de abstracción.

IV. Conclusión

El estudio llevado a cabo sostiene que, en el aprendizaje inicial de la programación, utilizar entornos basados en bloques se ha demostrado como una estrategia didáctica muy eficaz, puesto que al eliminar el factor sintáctico permite que los aprendices centren su esfuerzo cognitivo en la lógica algorítmica y en analizar la semántica de las estructuras de control. Esta ventaja conceptual persiste y se transfiere positivamente en el paso hacia lenguajes de programación basados en texto.

No obstante, la investigación muestra que el salto directo de bloques a código textual está limitado por el importante vacío técnico que se genera al aprender la sintaxis estricta de los lenguajes, y por la percepción que tienen los estudiantes sobre los bloques como si fueran poco auténticos, lo que puede reducir el interés de aprendizaje a largo plazo si no se aplica un apoyo adecuado.

Se concluye que el futuro de la educación informática no debe plantearse como una polarización total entre bloques o texto. El uso de herramientas intermedias como la edición basada en marcos y la incorporación de modelos predictivos de aprendizaje generarán entornos adaptativos que respeten las necesidades individuales de cada estudiante, promoviendo así una transición gradual y sostenible hacia la programación profesional.

Referencias

[1] D. Weintrop, "Block-based programming in computer science education," *Communications of the ACM*, vol. 62, no. 8, pp. 22–25, 2019. https://doi.org/10.1145/3341221

[2] D. Weintrop y U. Wilensky, "Comparing block-based and text-based programming in high school computer science classrooms," *ACM Transactions on Computing Education (TOCE)*, vol. 18, no. 1, art. 3, pp. 1–25, 2017. https://doi.org/10.1145/3089799

[3] F. M. Fahid *et al.*, "Trajectory-based novice student modeling for block-based programming," en *Proc. 29th ACM Conf. on User Modeling, Adaptation and Personalization (UMAP '21)*, 2021, pp. 189–200. https://doi.org/10.1145/3450613.3456833

[4] M. J. Gomez, M. Moresi y L. Benotti, “Text-based programming in primary school: A comparative study of programming abilities of children with and without block-based programming experience,” en *Proc. 2019 ACM Conf. on Innovation and Technology in Computer Science Education (ITiCSE ’19)*, 2019, pp. 402–408. https://doi.org/10.1145/3304221.3319734

[5] M. Kölling, N. C. C. Brown y A. Altadmri, “Frame-based editing: Easing the transition from blocks to text-based programming,” en *Proc. 10th Workshop in Primary and Secondary Computing Education (WIPSCE ’15)*, 2015, pp. 63–72. https://doi.org/10.1145/2818314.2818331